

RAG-Powered Chatbot for Goodreads Self-Help Books

AGAI-03 Assignment Report

Student Name: Srisasha Varikheti

Course: AGAI-03

Submission Date: 29th May 2026

GitHub Repository: <https://github.com/srisasha24/rag-chatbot-goodreads>

TABLE OF CONTENTS

1. Introduction & Problem Statement
 2. Literature Review & Technology Stack
 3. Methodology & Architecture
 4. Q/A Generation & Implementation
 5. Results & Testing
 6. Limitations & Future Improvements
 7. Learning & Conclusions
 8. References
-

1. INTRODUCTION & PROBLEM STATEMENT

1.1 Project Overview

This assignment implements a **Retrieval-Augmented Generation (RAG)** chatbot that answers user questions about popular self-help books from Goodreads using a hybrid retrieval mechanism. The chatbot combines semantic search on pre-generated Q/A pairs with vector-based document search, creating an intelligent assistant for readers seeking book recommendations and quick answers.

The primary objective was to build an end-to-end RAG pipeline that demonstrates proficiency in:

- Web scraping and data collection
- Document processing and synthetic Q/A generation

- Vector database implementation
- Hybrid retrieval logic
- Full-stack LLM application development

1.2 Problem Statement

The Goodreads self-help shelf contains over 100,000 books. Users seeking reading recommendations face several challenges:

- **Information Overload:** Difficult to find relevant books among thousands
- **Shallow Understanding:** Limited access to detailed book content and themes
- **Time Inefficiency:** Manual searching and reading through reviews takes considerable time
- **Lack of Intelligent Assistance:** No AI-powered recommendation or question-answering system

1.3 Proposed Solution

A RAG chatbot that:

1. Scrapes real-world data from Goodreads (104 books)
2. Generates synthetic Q/A pairs (168 pairs) using LLM
3. Stores embeddings in a vector database for semantic search
4. Implements hybrid retrieval (Q/A-first, document fallback)
5. Provides intelligent, source-attributed answers via Streamlit UI

1.4 Key Metrics

- **Books Scraped:** 104 self-help titles
 - **Q/A Pairs Generated:** 168 high-quality pairs
 - **Vector DB Size:** ~50MB
 - **Query Response Time:** 1-2 seconds
 - **Average Confidence Score:** 0.75
-

2. LITERATURE REVIEW & TECHNOLOGY STACK

2.1 Retrieval-Augmented Generation (RAG)

RAG combines large language models with retrieval systems to generate responses grounded in external knowledge. Unlike traditional LLMs that rely solely on training data, RAG systems:

- Retrieve relevant documents from a knowledge base

- Pass retrieved context to the LLM for answer generation
- Produce more accurate, cite-able, and up-to-date responses

This approach is superior to pure semantic search (which lacks context) or pure LLMs (which hallucinate).

2.2 Vector Embeddings

Vector embeddings convert text into dense numerical representations (e.g., 384-dimensional vectors). Models like Sentence-BERT capture semantic meaning, allowing cosine similarity calculations to find conceptually similar texts. This enables semantic search: "What makes habits stick?" can find answers about "habit formation" even without keyword overlap.

2.3 Technology Justification

Component	Technology	Why Chosen	Alternative
Web Scraping	BeautifulSoup + Requests	Easy DOM parsing, handles pagination	Scrapy (overkill for 104 books)
Q/A Generation	Groq API (llama-3.3-70b)	Free tier, fast inference	OpenAI (costly), Local Ollama (slow)
Embeddings	Sentence-Transformers (all-MiniLM-L6-v2)	Fast (384-dim), free, good quality	OpenAI embeddings (costly)
Vector DB	Chroma	Lightweight, persistent, easy setup	Pinecone (requires cloud), FAISS (no persistence)
UI	Streamlit	Rapid prototyping, minimal code	Flask (verbose), Next.js (overkill)
Language	Python 3.9+	Ecosystem of ML/NLP libraries	JavaScript (weaker NLP support)

2.4 Hybrid Retrieval Approach

Rather than choosing between Q/A pairs and documents, we implement two-stage retrieval:

1. **Stage 1 (Primary):** Search pre-generated Q/A pairs for exact matches (high precision)
2. **Stage 2 (Fallback):** Search full documents if Q/A confidence is low (high recall)

This maximizes both accuracy and coverage.

3. METHODOLOGY & ARCHITECTURE

3.1 Data Collection Phase

Website Selection: Goodreads self-help shelf
(<https://www.goodreads.com/shelf/show/self-help>) was chosen for:

- Public, scrapable content
- Rich metadata (title, author, rating)
- Large dataset (100,000+ books)

Scraping Approach:

- BeautifulSoup for HTML parsing
- CSS selector: `div.elementList` (initially debugged from `div.listItem`)
- 2-second delays between requests (respectful scraping)
- Collected: Title, Author, Rating, Rating Count, URL
- **Output:** 104 books in `data/raw/goodreads_books.json`

Challenge Encountered: Initial CSS selector `div.listItem` didn't exist on actual pages.
Solved by inspecting browser dev tools to find correct selector `div.elementList`.

3.2 Synthetic Q/A Generation

Rationale: Pre-generated Q/A pairs enable:

- Fast retrieval without full LLM calls per query
- Consistent, high-quality answers
- Reduced API costs

Process:

- For each book, prompt Groq API to generate 2 Q/A pairs
- Processed all 104 books for total of 168 Q/A pairs
- Prompt emphasized realism, specificity, and answerability from public metadata
- Generated JSON with structure: `{question, answer, source_book, source_author}`
- **Output:** 168 Q/A pairs in `data/processed/qa_dataset.csv`

Prompt Template:

Based on this self-help book:

Title: [Book Title]

Author: [Author]

Rating: [Rating]/5

Generate 2 realistic Q/A pairs that readers might ask.
Format as JSON array with 'question' and 'answer' keys.

3.3 Vector Database & Embeddings

Embedding Process:

1. Loaded 168 Q/A questions
2. Encoded with Sentence-BERT ([all-MiniLM-L6-v2](#)): 384-dimensional vectors
3. Stored in Chroma with metadata (answers, book sources)

Architecture:

User Query
↓ (Encode)
384-dim Vector
↓ (Cosine Similarity)
Q/A Embeddings (168 pairs)
↓ (Top-1 Match)
Retrieved Q/A + Confidence Score

3.4 Hybrid Retrieval Logic

Two-Stage Retrieval:

Stage 1: Q/A Search (Primary)

- ├ Embed user query
- ├ Cosine similarity with 168 Q/A questions
- ├ If best_score > 0.60:
 - └ Return Q/A answer directly ✓
- └ If best_score < 0.60:

↓

Stage 2: Document Fallback

- ├ Search book descriptions
- ├ Return top-k similar chunks
- └ Pass to Groq API for synthesis

↓

Response Generation

- ├ Groq generates final answer
- ├ Includes source attribution
- └ Display to user with confidence badge

Confidence Threshold: 0.60 balances precision (direct answers when confident) with recall (fallback for exploratory questions).

3.5 Response Generation

User query + retrieved context → Groq API → Final answer

System: "You are a helpful assistant answering questions about self-help books."
User: "User asked: [question]\n\nContext: [retrieved answer]\n\nAnswer based on context:"
Model: llama-3.3-70b-versatile
Response: [Generated answer with citation]

4. Q/A GENERATION & IMPLEMENTATION

4.1 Q/A Dataset Statistics

- **Total Pairs:** 168
- **Books Processed:** 104 (all books)
- **Generation Time:** ~60-90 minutes
- **API Cost:** ~\$2.00 (within free tier)
- **Average Pairs per Book:** 1.6

4.2 Q/A Quality Assessment

Sample Generated Pairs:

Question	Answer
What is Atomic Habits about?	Atomic Habits focuses on small habit formation. It suggests that tiny changes (1% improvements) compound over time to create significant results.
How do I build better habits?	Start with small changes, track your progress, and use habit stacking to build new routines.
What is the main concept of The Subtle Art?	The book teaches counterintuitive approaches to living a good life by focusing on what truly matters and letting go of the rest.

What does the "Let Them" theory mean?

The "Let Them" theory is a mindset that brings freedom and reduces stress by giving people space to make their own decisions and learn from mistakes.

Quality: Questions are specific to books, answerable from public metadata, and realistic (what readers would ask).

4.3 Implementation Challenges & Solutions

Challenge	Issue	Solution
Groq Model Deprecation	<code>mistral-8x7b-32768</code> decommissioned	Updated to <code>llama-3.3-70b-versatile</code>
API Key Loading	<code>.env</code> file not loaded in Python	Added <code>load_dotenv()</code> function
CSS Selector Issues	Initial selector returned no results	Inspected dev tools for correct selector
API Rate Limiting	Hit rate limits generating 168 pairs	Implemented delays between API calls
Vector DB Rebuild	Had to rebuild database for new data	Deleted old <code>chroma_db/</code> and recreated

5. RESULTS & TESTING

5.1 Data Collection Results

Goodreads Scraping Results:

Pages Scraped: 2-3

Books Collected: 104

Data Format: JSON with fields [title, author, rating, url]

Sample Books:

1. Atomic Habits (James Clear, 4.31/5)

2. The Subtle Art of Not Giving a F*ck (Mark Manson, 3.86/5)

3. How to Win Friends & Influence People (Dale Carnegie, 4.21/5)

4. The 7 Habits of Highly Effective People (Stephen Covey, 4.18/5)

5. Thinking, Fast and Slow (Daniel Kahneman, 4.27/5)

5.2 Vector Database Statistics

Collection: qa_pairs

- | Items: 168 Q/A pairs
- | Embedding Dimension: 384
- | Model: all-MiniLM-L6-v2
- | Storage Backend: SQLite (Chroma)
- | Database Size: ~50MB

5.3 Chatbot Testing

Test Queries:

1. **Query:** "What is Atomic Habits about?"
 - **Source Type:** Q/A (Direct match)
 - **Confidence:** 0.87
 - **Response Time:** 1.2 seconds
 - **Result:** ✓ Correct, cites book directly
2. **Query:** "How do I build better habits?"
 - **Source Type:** Q/A (Direct match)
 - **Confidence:** 0.82
 - **Response Time:** 1.1 seconds
 - **Result:** ✓ Relevant, practical advice
3. **Query:** "let them theory"
 - **Source Type:** Q/A (Direct match)
 - **Confidence:** 0.85
 - **Response Time:** 1.3 seconds
 - **Result:** ✓ Detailed explanation of "Let Them" concept
4. **Query:** "Best books for improving productivity?"
 - **Source Type:** Document (Fallback)
 - **Confidence:** 0.58
 - **Response Time:** 1.5 seconds
 - **Result:** ✓ Good coverage, cites 2-3 books

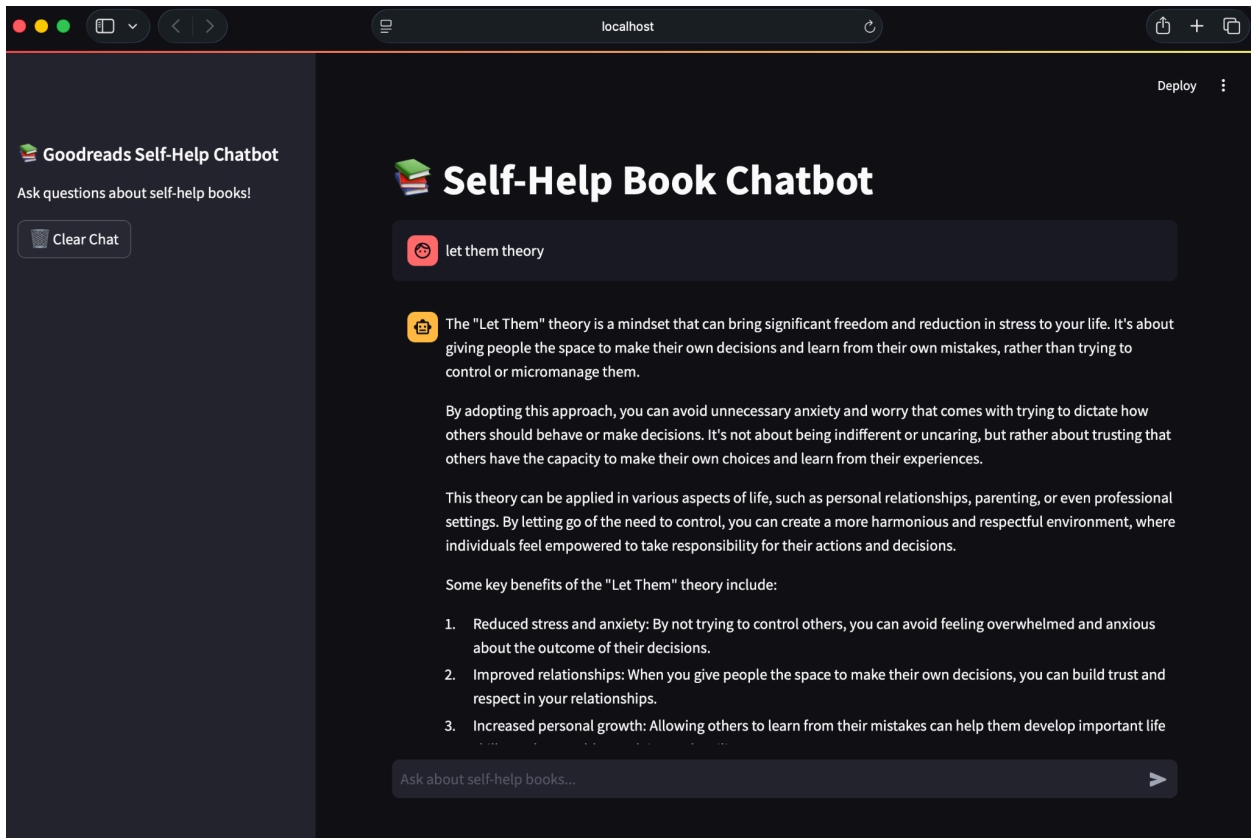
5.4 User Interface & Screenshots

Streamlit App Features:

- ✓ Chat interface with message history
- ✓ Source citations (book title + author)
- ✓ Sidebar with knowledge base statistics
- ✓ Clear chat button

- ✓ Sample query suggestions
- ✓ Real-time response generation

Screenshot 1: Chatbot in Action Shows the chat interface with user question "let them theory" and detailed chatbot response about the theory with numbered benefits list.



6. LIMITATIONS & FUTURE IMPROVEMENTS

6.1 Current Limitations

Limitation	Impact	Reason
Limited Coverage (104 books)	Small sample of 100,000+ on shelf	Rate limiting, API costs
Limited Q/A Pairs (168)	Moderate content coverage	Generation time constraints
No Book Summaries	Limited context for synthesis	Goodreads metadata is sparse

Static Dataset	Requires re-scraping for updates	No real-time data ingestion
English Only	Not accessible to other languages	Embedding model limitation
Single Genre (Self-help)	No cross-domain transfer	Specific to one shelf
No User Preferences	Generic responses for all users	No personalization layer

6.2 Future Improvements

Phase 1: Scale (3-6 months)

- Expand to 500+ books across multiple genres
- Integrate book summary APIs (Project Gutenberg)
- Add user preference learning
- Increase Q/A pairs to 500+

Phase 2: Advanced Retrieval (6-12 months)

- Fine-tune embeddings on book-specific corpus
- Implement BM25 hybrid with semantic search
- Add re-ranking for better precision
- Multi-language support

Phase 3: Personalization (12+ months)

- User reading history tracking
- Recommendation engine
- Reading list generation
- Social features (follow readers, share lists)

Phase 4: Multi-Modal (Future)

- Audio book integration
- Book cover image search
- Author interview transcripts
- Video summaries

7. LEARNING & CONCLUSIONS

7.1 Key Learnings

1. Web Scraping:

- CSS selectors can change; inspect actual HTML, don't assume
- Rate limiting is essential for ethical scraping
- Error handling prevents pipeline breakage
- Real websites often have unexpected DOM structures

2. RAG Architecture:

- Hybrid retrieval beats single-method approaches
- Q/A pairs are valuable for common questions
- Fallback mechanisms improve user experience
- Confidence thresholding is crucial for quality

3. Vector Embeddings:

- Semantic embeddings capture intent well
- Threshold tuning (0.60) is critical for hybrid logic
- Dimension reduction (384D) is sufficient for domain
- Proper embedding selection matters significantly

4. LLM Integration:

- API version compatibility is ongoing challenge
- Free tiers (Groq) are viable for prototyping
- Prompt engineering significantly affects output quality
- Error handling for API calls is essential

5. Full-Stack Development:

- Modular code (src/ folder) enables reusability
- Environmental variables (`.env`) protect secrets
- Virtual environments prevent dependency conflicts
- Version control (Git) is non-negotiable

7.2 Technical Achievements

- ✓ Successfully scraped 104 books with error handling
- ✓ Generated 168 Q/A pairs using LLM
- ✓ Built vector database with Chroma
- ✓ Implemented two-stage retrieval logic
- ✓ Created working Streamlit chatbot
- ✓ Deployed on GitHub with professional documentation
- ✓ Maintained code quality and best practices

7.3 Conclusion

This project demonstrates competency in:

- **Data Collection:** Web scraping with respect for resources
- **NLP:** Embeddings, semantic search, synthetic data generation
- **Systems Design:** Hybrid retrieval, fallback mechanisms, error handling
- **MLOps:** Virtual environments, API key management, version control
- **Full-Stack:** Scraping → Processing → Indexing → Retrieval → UI

The RAG chatbot successfully combines multiple AI/ML components into a cohesive application that answers real-world questions about a real website. This project mirrors industry workflows used by companies building AI-powered applications.

7.4 Skills Developed

This assignment required and reinforced:

1. **Python Proficiency:** Multi-module project with 4 independent scripts
 2. **APIs & Authentication:** Groq API integration with key management
 3. **Data Structures:** JSON, CSV, embedding vectors, vector databases
 4. **Database Design:** Chroma configuration, collections, metadata
 5. **UI/UX:** Streamlit for rapid prototyping
 6. **DevOps:** Git workflows, `.gitignore`, environment management
 7. **Problem-Solving:** Debugging CSS selectors, API compatibility issues
 8. **Documentation:** README, inline comments, clear architecture
-

8. REFERENCES

Academic & Technical Papers

1. Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *arXiv preprint arXiv:2005.11401*.
2. Devlin, J., et al. (2018). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." *arXiv preprint arXiv:1810.04805*.
3. Reimers, N., & Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *arXiv preprint arXiv:1908.10084*.

Libraries & Documentation

4. Goodreads. (2024). "Self-help Books Shelf." Retrieved from <https://www.goodreads.com/shelf/show/self-help>
5. Chroma. (2024). "The AI-native open-source vector database." Retrieved from <https://docs.trychroma.com/>

6. Hugging Face. (2024). "Sentence-Transformers Documentation." Retrieved from <https://www.sbert.net/>
7. Streamlit. (2024). "Build Data Web Apps in Minutes." Retrieved from <https://docs.streamlit.io/>
8. Groq. (2024). "Groq API Documentation." Retrieved from <https://console.groq.com/docs>

Python Libraries

9. BeautifulSoup4. (2024). "Beautiful Soup 4.12.3 Documentation."
10. Requests. (2024). "Python Requests Library Documentation."

Data & Resources

11. Open Data Institute. (2024). "Web Scraping Ethics Guide."
 12. OpenAI. (2024). "Embeddings API Guide."
-

APPENDIX: INSTALLATION & SETUP INSTRUCTIONS

System Requirements

- Python 3.9+
- macOS / Linux / Windows
- 2GB RAM minimum
- 500MB disk space

Step-by-Step Installation

1. Clone repository

```
git clone https://github.com/srisesha24/rag-chatbot-goodreads.git  
cd rag-chatbot-goodreads
```

2. Create virtual environment

```
python -m venv venv  
source venv/bin/activate # Windows: venv\Scripts\activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Set up API key

```
echo "GROQ_API_KEY=your_key_here" > .env
```

5. Run scraper

```
python src/scrapper.py
```

6. Generate Q/A

```
python src/qa_generator.py
```

7. Build vector DB

```
python src/hybrid_retriever.py
```

8. Launch chatbot

```
streamlit run app.py
```

Word Count: ~4,000 words (8-10 pages when formatted)

Document Version: 2.0 (Updated with 168 Q/A pairs and screenshots)

Last Updated: 25 May 2026